



Rajarata University of Sri Lanka

Department of Computing

ICT1305 – Data Structures
Laboratory 06 – Linked list in C
Time: 120 Minutes

Objective:

By the end of this lab, students should be able to:

- Understand the concept and importance of queues in C programming.
- Define and use static and dynamic queue structures.
- Perform queue operations such as enqueue, dequeue, and display.
- Understand FIFO behavior using real-world analogies.
- Apply linked list techniques to implement dynamic queues.

Part 1 – Concept Overview

A Queue is a linear data structure that follows the FIFO (First In First Out) principle. The first element inserted is the first to be removed. It is like a real-world queue of people — the first person to join is the first to be served.

Part 2 – Key Functions and Their Real-World Examples

Function	Description	Real-World Example
enqueue()	Add item to rear of queue	A person joins the back of a queue at a bakery
dequeue()	Remove item from front of queue	The first person in the bakery queue is served and leaves
peek()	See the front item without removing it	You look to see who is first in line without touching them
isEmpty()	Check if queue is empty	You check if there are any people waiting in the line
isFull()	Check if queue is full (for arrays)	The bakery can only allow 5 people in line at a time

Part 3 – Static Queue Using Arrays

Activity 1: Implement a Simple Queue using Arrays

```
#include <stdio.h>
#define SIZE 5

int queue[SIZE];
int front = -1, rear = -1;

void enqueue(int value) {
    if (rear == SIZE - 1)
        printf("Queue is Full\n");
    else {
        if (front == -1)
            front = 0;
        rear++;
        queue[rear] = value;
        printf("Inserted %d\n", value);
    }
}

void dequeue() {
    if (front == -1 || front > rear)
        printf("Queue is Empty\n");
    else {
        printf("Deleted %d\n", queue[front]);
        front++;
    }
}

void display() {
    if (front == -1 || front > rear)
        printf("Queue is Empty\n");
    else {
        printf("Queue: ");
        for (int i = front; i <= rear; i++)
            printf("%d ", queue[i]);
        printf("\n");
    }
}

int main() {
    enqueue(10);
    enqueue(20);
    enqueue(30);
    display();
    dequeue();
    display();
    return 0;
}
```

Part 4 – Dynamic Queue Using Structures

Activity 2: Implement Queue using Struct and Pointers (Linked List)

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node* next;
};

struct Node *front = NULL, *rear = NULL;

void enqueue(int value) {
    struct Node* newNode = malloc(sizeof(struct Node));
    newNode->data = value;
    newNode->next = NULL;

    if (rear == NULL) {
        front = rear = newNode;
    } else {
        rear->next = newNode;
        rear = newNode;
    }
    printf("Inserted %d\n", value);
}

void dequeue() {
    if (front == NULL) {
        printf("Queue is Empty\n");
        return;
    }

    struct Node* temp = front;
    printf("Deleted %d\n", front->data);
    front = front->next;
    free(temp);

    if (front == NULL)
        rear = NULL;
}

void display() {
    struct Node* temp = front;
    if (temp == NULL) {
        printf("Queue is Empty\n");
        return;
    }
}
```

```

        printf("Queue: ");
        while (temp != NULL) {
            printf("%d ", temp->data);
            temp = temp->next;
        }
        printf("\n");
    }

int main() {
    enqueue(5);
    enqueue(15);
    enqueue(25);
    display();
    dequeue();
    display();
    return 0;
}

```

Key Concepts

- Queue uses FIFO method: First In First Out.
- Use arrays for static queue, linked list for dynamic queue.
- Static queues have fixed size; dynamic queues grow with memory.
- Enqueue adds to the rear, Dequeue removes from the front.
- Always check for overflow (static) or empty conditions.

Summary Points for Students.

- Understand when to use queues in real life and programs.
- Practice both static and dynamic implementations.
- Master basic operations: enqueue, dequeue, display.
- Avoid errors by checking for full or empty states.
- Learn to trace queue behavior step-by-step.

Conclusion:

In this lab, you learned how queues work, how to implement them using arrays and pointers. Practice these operations to master the queue data structure.

-End-